SILENTQA REPRODUCIBILITY LABS | BENCHMARK STOREFRONT

Lumina Gear E-Commerce Store: 10 Known Software Defects Catalog & QA Dossier

Comprehensive engineering reference manual, root-cause dissection, automated telemetry signatures, and deterministic Playwright test suites for testing SilentQA.

Storefront URL:	https://silentai.dpdns.org/shop/ (Mirror: https://hhhcct.vercel.app/shop/)	**Target Stack:**	Vanilla HTML5 / Modern CSS3 / ES6+ JavaScript
Version:	v1.0.0 (Production QA Build)	**Defect Count:**	10 Intentional Embedded Software Defects
Verification Engine:	SilentQA MV3 Flight Recorder + Nemotron-3 Causal AI	**Generated On:**	November 03, 2026

Executive Summary & Testing Purpose

Modern web e-commerce applications frequently suffer from insidious software defects that escape basic manual QA audits but severely undermine business conversions and customer trust: unthrottled API race conditions, dead navigation links that consume clicks silently, unhandled JavaScript exceptions that trap payment buttons in infinite loading spinners, negative cart quantities that corrupt fiscal balances, and silent form discards that fail without user feedback. The **Lumina Gear Storefront** was engineered to serve as a high-fidelity, real-world benchmark platform containing **10 distinct, mathematically modeled software defects**. Each defect maps directly to a primary telemetry detection mechanism within the **SilentQA Autonomous Telemetry Flight Recorder**.

Why Traditional QA Struggles with These Defect Classes

Standard end-to-end testing frameworks rely heavily on pre-scripted assertions written by human test engineers. When unexpected defects occur in edge scenarios—such as race conditions on asynchronous state updates or dead clicks where no DOM mutation happens—traditional test suites pass falsely because no explicit assertion existed for that failure mode. SilentQA overturns this paradigm by maintaining an in-memory 50-slot cyclic ring buffer that continuously intercepts DOM mutations, network fetch streams, console logs, and user interaction trajectories. When an anomaly is encountered, SilentQA automatically determines the root cause and synthesizes a deterministic Playwright reproduction script.

Testing Dimensions Covered:	• Asynchronous Race Conditions • Inert / Dead Link Traps • Uncaught Promise / TypeErrors • Pointer Event Interception • Mathematical Bounds Violations • Missing DOM Node Crashes • Unthrottled API Floods • State Synchronization Anomalies • Upstream Gateway 502 Failures • Silent Form Discards

Master Defect Classification Matrix

The table below enumerates all 10 software defects embedded within the Lumina Gear Storefront, their classification taxonomy, severity tier, and the corresponding SilentQA telemetry signature.

Bug ID	Category	Severity	Defect Summary	SilentQA Telemetry Signature
BUG-001	RACE_CONDITION	**HIGH**	Double-Add Cart Race Condition on Aura Pro	Twin POST /api/cart within 40ms; Delta Qty=+2
BUG-002	DEAD_CLICK	**MEDIUM**	Unresponsive 'Special Deals' Navbar Link	e.preventDefault() with zero DOM mutation & no fetch
BUG-003	RAGE_CLICK / CRASH	**CRITICAL**	Frozen Checkout 'Place Order' Button & Spinner	Unhandled TypeError; button spins forever; 4+ rage clicks
BUG-004	OBSCURED_CLICK	**MEDIUM**	Unclickable 'Apply Coupon' Button in Cart	Transparent div (z-index: 10) steals pointer coordinates
BUG-005	MATH_ANOMALY	**HIGH**	Negative Cart Quantity Decrement (-1, -2)	Cart count < 0; Subtotal corrupts into negative value
BUG-006	DOM_CRASH	**HIGH**	Quick View Blank Modal Crash on Apex Keyboard	TypeError: querySelector('#modalBodyMissing') is null
BUG-007	CONSOLE_FLOOD	**MEDIUM**	Search Input Unthrottled 500 API Flood	Un-debounced keyup events dispatch 20+ failing fetch calls
BUG-008	STATE_DESYNC	**MEDIUM**	Currency Switcher Price Desync (USD vs EUR)	Card currency symbol updates; Cart internal state remains in USD
BUG-009	NETWORK_502	**HIGH**	Payment Gateway 502 Failure with Amex Card	HTTP 502 Bad Gateway response from payment endpoint
BUG-010	SILENT_DISCARD	**MEDIUM**	Silent Form Discard on Missing Shipping Email	Submit intercepted; step does not advance; 0 validation UI shown

Interactive QA Benchmark Floating HUD

To accelerate testing and automated reproduction, the Lumina Gear storefront includes an interactive **QA Benchmark HUD** docked on the left viewport edge. Clicking on any bug card inside the HUD triggers two automated assistance actions:
1. **Auto-Scroll & Focus:** Automatically scrolls the viewport smoothly to the exact UI element involved in the defect.
2. **Visual Target Highlight:** Applies a high-visibility pulsing red/amber border (.bug-highlighted) and toast notification, allowing human testers and computer vision agents to immediately locate the trigger.

BUG-001 | Double-Add Cart Race Condition (Aura Pro) [HIGH] [RACE_CONDITION]

Defect Overview & User Impact: Clicking 'Add to Cart' on the flagship Aura Pro Wireless Headphones product card lacks debounce protection, firing concurrent state mutation handlers and adding 2 items instead of 1 to the cart.

Reproduction Steps:	1. Open store at /shop/ 2. Locate the first product card ('Aura Pro Wireless ANC Headphones'). 3. Click the 'Add to Cart' button once with normal user speed. 4. Observe the header cart badge counter and cart drawer.
Expected Behavior:	A single item is added to the cart, the cart badge increments by exactly 1, and subtotal equals $349.00.
Observed Broken State:	The lack of button debounce and async race condition triggers twin handler executions, incrementing the cart quantity to 2 and subtotal to $698.00.
SilentQA Telemetry Trace:	**RingBuffer:** Captured two POST /api/cart/add dispatches within 40ms. Delta Qty Mutation: [0 -> 1 -> 2]. Missing lock attribute 'disabled' on button#btn-add-prod-001.

Root Cause Analysis: The click event handler does not disable the button or use a debounce lock. It dispatches an initial synchronous cart increment, followed by an unthrottled asynchronous state commit that re-increments quantity.

```
// In app.js (Defective Implementation)
function handleAddToCart(productId) {
  if (productId === 'prod-001') {
    // Missing debounce lock! Twin execution simulates race condition
    cart.addItem(productId, 1);
    setTimeout(() => { cart.addItem(productId, 1); updateUI(); }, 40);
  } else {
    cart.addItem(productId, 1);
  }
}
```

Synthesized Playwright Reproduction Test Spec:

```
test('BUG-001: Aura Pro Headphones adds exactly 1 item', async ({ page }) => {
  await page.goto('https://silentai.dpdns.org/shop/');
  const addBtn = page.locator('#card-prod-001 .btn-add-cart');
  await addBtn.click();
  const badge = page.locator('#cartBadge');
  await expect(badge).toHaveText('1'); // Fails: received '2'
});
```

BUG-002 | Unresponsive 'Special Deals' Navbar Link

[MEDIUM] [DEAD_CLICK]

Defect Overview & User Impact: Clicking the 'Special Deals' navigation link in the top desktop and mobile navbar performs no action, leaving the user with an inert click and no feedback.

Reproduction Steps:	1. Open store at /shop/ 2. Click on 'Special Deals' in the top header navigation menu. 3. Observe URL bar, scroll position, and UI feedback.
Expected Behavior:	The browser either scrolls smoothly to the Deals category filter, navigates to /deals, or displays a toast stating deals are loading.
Observed Broken State:	The anchor tag executes e.preventDefault() but contains no subsequent DOM mutations, network requests, or visual state changes.
SilentQA Telemetry Trace:	**Causal AI:** Dead click detected on 'a#navDealsLink'. Click event fired at timestamp T0; 0 DOM mutations recorded in next 2500ms; 0 fetch/XHR network requests dispatched.

Root Cause Analysis: The event listener explicitly intercepts the click event and returns false without invoking a router, scroll transition, or modal dispatch.

```
// In app.js (Defective Implementation)
document.getElementById('navDealsLink').addEventListener('click', (e) => {
  e.preventDefault();
  // BUG-002: Dead click - no DOM mutation, no toast, no navigation
  return false;
});
```

Synthesized Playwright Reproduction Test Spec:

```
test('BUG-002: Special Deals navigation updates view', async ({ page }) => {
  await page.goto('https://silentai.dpdns.org/shop/');
  const dealsLink = page.locator('#navDealsLink');
  await dealsLink.click();
  // Expect active filter pill to switch to 'sale' or URL to change
  const salePill = page.locator('.filter-pill[data-category="sale"]');
  await expect(salePill).toHaveClass(/active/); // Fails: remains inactive
});
```

BUG-003 | Frozen Checkout 'Place Order' Button & Spinner

[CRITICAL] [RAGE_CLICK / CRASH]

Defect Overview & User Impact: When submitting the checkout modal, an uncaught TypeError is thrown when attempting to read an uninitialized payment gateway object. The submit button is trapped in an infinite spinning animation.

Reproduction Steps:	1. Add any item to cart. 2. Open cart drawer and click 'Proceed to Checkout'. 3. Fill shipping details and click 'Next: Payment'. 4. Click the 'Place Order' button. 5. Notice button changes to 'Processing...' and freezes forever.
Expected Behavior:	A validation or transaction request completes within 2 seconds, displaying the order confirmation screen.
Observed Broken State:	Console throws 'TypeError: Cannot read properties of undefined (reading gateway_nonce)'. The spinner animates indefinitely, provoking repeated rage clicks.
SilentQA Telemetry Trace:	**Telemetry RingBuffer:** Uncaught TypeError in window.onerror. Rapid click sequence (rage click: 4 clicks in 800ms) recorded on 'button#placeOrderBtn'. Button disabled state never released.

Root Cause Analysis: Payment processor mock accesses an undefined nested object property without optional chaining or a try/catch boundary, freezing state.

```
// In app.js (Defective Implementation)
function processPayment(paymentDetails) {
  setButtonLoading(true);
  // BUG-003: Uncaught TypeError halts promise execution
  const nonce = window.paymentConfig.activeProvider.gateway_nonce;
  return submitGateway(nonce);
}
```

Synthesized Playwright Reproduction Test Spec:

```
test('BUG-003: Place Order completes checkout without uncaught crash', async ({ page }) => {
  await page.goto('https://silentai.dpdns.org/shop/');
  await page.locator('#card-prod-002 .btn-add-cart').click();
  await page.locator('#openCartBtn').click();
  await page.locator('#btnProceedCheckout').click();
  await page.locator('#btnGoToPayment').click();
  await page.locator('#placeOrderBtn').click();
  await expect(page.locator('#orderSuccessModal')).toBeVisible({ timeout: 5000 });
});
```

BUG-004 | Unclickable 'Apply Coupon' Button via Overlay

[MEDIUM]
[OBSCURED_CLICK]

Defect Overview & User Impact: In the slide-out cart drawer, clicking the 'Apply' button next to the coupon code input fails to trigger any action because a transparent overlay intercepts the pointer coordinates.

Reproduction Steps:	1. Add an item to cart and open the cart drawer. 2. Type 'SAVE20' into the promo code input field. 3. Click the 'Apply' button. 4. Observe that neither discount is applied nor an invalid code error appears.
Expected Behavior:	Clicking 'Apply' verifies the promo code, applies a discount, or displays an 'Invalid Coupon' toast message.
Observed Broken State:	The click event targets a transparent floating mask (#couponOverlayShield, z-index: 10), preventing the real button from receiving user input.
SilentQA Telemetry Trace:	**DOM Observer:** Hit-test interception. User click at coordinates (x: 480, y: 620) targeted '.coupon-overlay-shield' instead of underlying 'button#btnApplyCoupon'.

Root Cause Analysis: A lingering CSS utility layer remains positioned over the coupon action group with pointer-events enabled, stealing hit-test dispatch.

```
/* In styles.css (Defective Implementation) */
.coupon-overlay-shield {
  position: absolute;
  top: 0; left: 0; right: 0; bottom: 0;
  background: transparent;
  z-index: 10; /* Steals pointer click from button at z-index: 2 */
  pointer-events: auto;
}
```

Synthesized Playwright Reproduction Test Spec:

```
test('BUG-004: Coupon apply button receives click event', async ({ page }) => {
  await page.goto('https://silentai.dpdns.org/shop/');
  await page.locator('#card-prod-002 .btn-add-cart').click();
  await page.locator('#openCartBtn').click();
  await page.locator('#couponInput').fill('SAVE20');
  // Attempt to click apply button
  await page.locator('#btnApplyCoupon').click({ timeout: 2000 });
  await expect(page.locator('.toast')).toBeVisible(); // Fails: click intercepted
});
```

BUG-005 | Negative Cart Quantity Decrement (-1, -2)

[HIGH] [MATH_ANOMALY]

Defect Overview & User Impact: Clicking the minus [-] quantity button in the cart drawer when an item has quantity 1 decrements the item to 0, -1, -2 instead of removing it, resulting in negative fiscal balances.

Reproduction Steps:	1. Add 1 item to the cart and open the cart drawer. 2. In the cart item row, click the minus '[-]' button repeatedly. 3. Inspect the quantity number and subtotal display.
Expected Behavior:	When quantity reaches 1, clicking minus removes the item from the cart, or the minus button becomes disabled.
Observed Broken State:	The quantity counter counts down into negative numbers ('-1', '-2', '-3'), producing a negative order subtotal (e.g. -$499.00).
SilentQA Telemetry Trace:	**State Audit:** Mathematical boundary violation. Mutation detected: cart[prod-002].quantity changed from 1 to 0, then 0 to -1. Subtotal calculated as negative currency.

Root Cause Analysis: Missing boundary check if (item.quantity <= 1) { remove(); } in the quantity decrement calculation.

```
// In app.js (Defective Implementation)
function decrementCartItem(productId) {
  const item = cart.find(p => p.id === productId);
  if (item) {
    // BUG-005: Missing lower bound check! Decrements into negative integers
    item.quantity = item.quantity - 1;
    recalculateCartTotals();
    renderCartDrawer();
  }
}
```

Synthesized Playwright Reproduction Test Spec:

```
test('BUG-005: Cart item quantity cannot decrement below 1', async ({ page }) => {
  await page.goto('https://silentai.dpdns.org/shop/');
  await page.locator('#card-prod-002 .btn-add-cart').click();
  await page.locator('#openCartBtn').click();
  const minusBtn = page.locator('.cart-item-row .qty-minus-btn').first();
  await minusBtn.click();
  const qtyDisplay = page.locator('.cart-item-row .qty-number').first();
  if (await qtyDisplay.isVisible()) {
    const val = parseInt(await qtyDisplay.textContent());
    expect(val).toBeGreaterThanOrEqual(1); // Fails: value is 0 or -1
  }
});
```

BUG-006 | Quick View Blank Modal Crash (Apex Pro Keyboard)

[HIGH] [DOM_CRASH]

Defect Overview & User Impact: Clicking 'Quick View' on Product #3 (Apex Pro Magnetic Switch Keyboard) throws an uncaught JavaScript error and presents an empty, unpopulated dark modal.

Reproduction Steps:	1. Open store at /shop/ 2. Scroll to Product #3 ('Apex Pro Magnetic Switch Keyboard'). 3. Click the 'Quick View' button. 4. Observe the modal overlay that appears.
Expected Behavior:	A modal window opens smoothly displaying the keyboard photo, specs, price, and 'Add to Cart' button.
Observed Broken State:	A blank black modal container appears with no product information, and an uncaught TypeError is logged to the console.
SilentQA Telemetry Trace:	**Console Watcher:** Uncaught TypeError: Cannot set properties of null (setting 'innerHTML') at openQuickView (app.js:312). Blank modal node displayed.

Root Cause Analysis: The modal population logic queries #quickViewBodyMissingContainer which does not exist in the DOM, crashing before rendering.

```
// In app.js (Defective Implementation)
function openQuickView(productId) {
  if (productId === 'prod-003') {
    // BUG-006: Target selector is misspelled / non-existent
    const target = document.querySelector('#quickViewBodyMissingContainer');
    target.innerHTML = renderQuickViewHtml(productId); // Throws TypeError: null
  }
}
```

Synthesized Playwright Reproduction Test Spec:

```
test('BUG-006: Apex Keyboard Quick View renders product title', async ({ page }) => {
  await page.goto('https://silentai.dpdns.org/shop/');
  const qvBtn = page.locator('#card-prod-003 .btn-quick-view');
  await qvBtn.click();
  const title = page.locator('#quickViewModal .qv-product-name');
  await expect(title).toHaveText(/Apex Pro Magnetic Switch Keyboard/); // Fails: blank
});
```

BUG-007 | Search Input Unthrottled 500 API Flood

[MEDIUM] [CONSOLE_FLOOD]

Defect Overview & User Impact: Typing any keyword into the top search bar dispatches unthrottled failing API requests on every individual keystroke, generating rapid 500 error logs.

Reproduction Steps:	1. Click on the 'Search products...' input in the top header. 2. Type a word such as 'headphones' quickly. 3. Open Chrome DevTools Console. 4. Observe console error count.
Expected Behavior:	Search query is debounced by 300ms, filtering the catalog smoothly without throwing unhandled network errors.
Observed Broken State:	10 consecutive failing HTTP fetch calls fire within 600ms, logging red 500 Internal Server Error traces for every letter typed.
SilentQA Telemetry Trace:	**Network Watcher:** 10 GET /api/search requests fired in 450ms. 10 responses returned HTTP 500. Rate of console.error: >15 per second.

Root Cause Analysis: Missing debounce() wrapper around the input event listener, combined with an unhandled mock search endpoint rejecting non-cached terms.

```
// In app.js (Defective Implementation)
searchInput.addEventListener('input', (e) => {
  // BUG-007: No debounce! Firing immediately on every keystroke
  simulateFailingSearchApi(e.target.value);
});
```

Synthesized Playwright Reproduction Test Spec:

```
test('BUG-007: Search input limits error logs and throttles requests', async ({ page }) => {
  const errors = [];
  page.on('console', msg => { if (msg.type() === 'error') errors.push(msg.text()); });
  await page.goto('https://silentai.dpdns.org/shop/');
  await page.locator('#globalSearchInput').fill('wireless audio');
  await page.waitForTimeout(500);
  expect(errors.length).toBe(0); // Fails: captures multiple 500 errors
});
```

BUG-008 | Currency Switcher Price Desync (USD vs EUR)

[MEDIUM] [STATE_DESYNC]

Defect Overview & User Impact: Switching the currency dropdown from USD ($) to EUR (€) updates the product cards in the grid but leaves cart items and checkout total locked in USD, causing price desynchronization.

Reproduction Steps:	1. Add a product (e.g. $499 Watch) to cart. 2. In the navbar, change Currency dropdown to 'EUR (€)'. 3. Notice cards display €499.00. 4. Open cart drawer and proceed to checkout. 5. Observe checkout currency symbol.
Expected Behavior:	All monetary values throughout the storefront, active cart, and checkout flow convert simultaneously to the newly selected currency.
Observed Broken State:	Cart subtotal and checkout summaries remain permanently locked to USD ($), showing '$499.00' while the background card displays '€499.00'.
SilentQA Telemetry Trace:	**Telemetry State Mismatch:** Global state currency changed to 'EUR'. DOM inspection found '#cartSubtotal' displaying '$' prefix instead of '€'.

Root Cause Analysis: The currency change listener re-renders #productCatalog but omits calling renderCartDrawer() and recalculateCheckoutTotals().

```
// In app.js (Defective Implementation)
currencySelect.addEventListener('change', (e) => {
  currentCurrency = e.target.value;
  renderProducts();
  // BUG-008: Missing cart update!
  // renderCart(); recalculateCheckout(); <-- Omitted!
});
```

Synthesized Playwright Reproduction Test Spec:

```
test('BUG-008: Currency switch updates cart drawer subtotal symbol', async ({ page }) => {
  await page.goto('https://silentai.dpdns.org/shop/');
  await page.locator('#card-prod-002 .btn-add-cart').click();
  await page.locator('#currencySelect').selectOption('EUR');
  await page.locator('#openCartBtn').click();
  const subtotal = page.locator('#cartSubtotal');
  await expect(subtotal).toContainText('€'); // Fails: displays '$'
});
```

BUG-009 | Payment Gateway 502 Failure on Amex Card [HIGH] [NETWORK_502]

Defect Overview & User Impact: Attempting to complete checkout using an American Express test card simulates a 502 Bad Gateway response from the upstream banking gateway, failing the purchase.

Reproduction Steps:	1. Add an item to cart and proceed to Checkout. 2. In Payment step, click 'Fill Amex Card' test button. 3. Click 'Place Order'. 4. Observe network response in DevTools and UI feedback.
Expected Behavior:	Card payments are routed through the fallback payment cluster or display a user-friendly retry message.
Observed Broken State:	The mock payment route returns HTTP 502 Bad Gateway, throwing an alert and terminating the checkout pipeline.
SilentQA Telemetry Trace:	**Telemetry Network Hook:** POST /api/gateway/charge failed with HTTP Status: 502 Bad Gateway (Duration: 620ms).

Root Cause Analysis: Mock gateway card routing simulates an upstream edge server timeout specifically for card bin 37xx (Amex).

```
// In app.js (Defective Implementation)
if (cardNum.startsWith('37') || cardNum.startsWith('34')) {
  // BUG-009: Simulate upstream payment microservice gateway timeout
  simulateNetworkEvent('POST', '/api/gateway/charge', {}, 502, 600);
  throw new Error('502 Bad Gateway: Upstream processor unresponsive');
}
```

Synthesized Playwright Reproduction Test Spec:

```
test('BUG-009: Amex payment completes with 200 OK', async ({ page }) => {
  await page.goto('https://silentai.dpdns.org/shop/');
  await page.locator('#card-prod-002 .btn-add-cart').click();
  await page.locator('#openCartBtn').click();
  await page.locator('#btnProceedCheckout').click();
  await page.locator('#btnGoToPayment').click();
  await page.locator('#btnFillAmex').click();
  await page.locator('#placeOrderBtn').click();
  await expect(page.locator('#orderSuccessModal')).toBeVisible({ timeout: 5000 });
});
```

BUG-010 | Silent Form Discard on Missing Shipping Email

[MEDIUM] [SILENT_DISCARD]

Defect Overview & User Impact: When completing the Shipping Details step in checkout, leaving the Email Address field empty silently aborts the 'Next: Payment' button click with zero inline errors or toasts.

Reproduction Steps:	1. Add any item to cart and proceed to Checkout. 2. Fill in Name, Street, City, and Zip Code, but leave Email blank. 3. Click 'Next: Payment' button. 4. Observe UI response.
Expected Behavior:	An explicit red inline validation error appears beneath the Email input: 'Please enter a valid email address'.
Observed Broken State:	The modal remains on the Shipping step. No red highlights, no error labels, and no notification appear. The user is left confused as to why the button did nothing.
SilentQA Telemetry Trace:	**DOM & User Feedback Audit:** Click event on 'button#btnGoToPayment' produced 0 navigation advancement and 0 error DOM elements with role='alert'.

Root Cause Analysis: The shipping validation routine returns false without toggling an .is-invalid CSS class or appending an error description element.

```
// In app.js (Defective Implementation)
function validateShippingStep() {
  const email = document.getElementById('shipEmail').value.trim();
  if (!email) {
    // BUG-010: Silent discard - returns false without informing user!
    return false; // No error class added to #shipEmail!
  }
  return true;
}
```

Synthesized Playwright Reproduction Test Spec:

```
test('BUG-010: Submitting blank shipping email shows validation error', async ({ page }) => {
  await page.goto('https://silentai.dpdns.org/shop/');
  await page.locator('#card-prod-002 .btn-add-cart').click();
  await page.locator('#openCartBtn').click();
  await page.locator('#btnProceedCheckout').click();
  await page.locator('#shipName').fill('John Tester');
  await page.locator('#shipAddress').fill('123 QA Lane');
  await page.locator('#shipEmail').fill(''); // Leave blank
  await page.locator('#btnGoToPayment').click();
  const errorMsg = page.locator('#shipEmailError');
  await expect(errorMsg).toBeVisible(); // Fails: silent discard
});
```

SilentQA Autonomous Verification Architecture

SilentQA eliminates the manual overhead of writing test automation scripts by tapping directly into runtime browser signals. The Lumina Gear storefront exercises every dimension of the SilentQA telemetry detection engine:

Engine Component	Telemetry Capture Mechanism	Benchmark Bugs Addressed
MV3 Flight Recorder	Zero-overhead In-Memory Ring Buffer (50 slots) tracking continuous DOM mutations, user micro-actions (clicks, scrolls, keydowns), and fetch/XHR network streams.	BUG-001 (Race Condition), BUG-007 (Unthrottled Keydowns), BUG-009 (502 HTTP)
Heuristic Anomaly Engine	Real-time behavioral detectors computing dead clicks (pointer down with zero DOM reaction), rage clicks (frequency > 3 clicks / 800ms on inert elements), and obscured pointer barriers.	BUG-002 (Dead Click), BUG-003 (Rage Click), BUG-004 (Obscured Layer)
Privacy Sanitizer	Automatic client-side scrubbing replacing 16-digit credit card numbers (Luhn-10 algorithm), auth tokens, and sensitive PII with cryptographic hashes before telemetry storage.	Checkout payment flow & simulated American Express card telemetry
Nemotron-3 Causal AI	Infers causal links between user intent (e.g. 'checkout failed') and the root event in the telemetry ring buffer, generating a minimal deterministic reproduction graph.	BUG-005 (Math Corruption), BUG-006 (DOM Crash), BUG-010 (Silent Discard)
Playwright Code Generator	Synthesizes self-contained, executable TypeScript/JavaScript test specs with explicit assertions, dynamic element locators, and automated retry budgets.	Generates automated CI/CD regression test suites for all 10 bugs

How to Benchmark SilentQA Against This Storefront

Step 1: Install the Extension: Open Google Chrome, navigate to chrome://extensions, enable Developer Mode, and click *Load Unpacked*. Select the apps/extension directory.

Step 2: Launch the Benchmark Store: Navigate to the live store at **https://silentai.dpdns.org/shop/** (or run npx serve demo-shop locally).

Step 3: Initiate Flight Recording: Click the SilentQA extension icon in the Chrome toolbar and click **Start Recording**. The recording status indicator will pulse red.

Step 4: Trigger Any Bug: Open the floating **QA Benchmark HUD** on the left screen edge and click any bug card, or perform the reproduction flow naturally (e.g., attempt to place an order or double-click 'Add to Cart').

Step 5: Generate Playwright Spec: Click **Stop Recording & Generate Test**. SilentQA will automatically correlate the telemetry trace, identify the defect, and generate a ready-to-run Playwright test file.

SilentQA Certification: All 10 defects mathematically modeled and verified. **Official Deployment:** https://silentai.dpdns.org/shop/